AI Workload Acceleration through Semantic Preserving Hardware Generation

Md Mahfuzul Haque, Md Rubel Ahmed
Louisiana Tech University
Ruston, USA
{mha059, mahmed}@latech.edu

***Abstract*—The growing demand for workload-aware machine-learning accelerators has renewed interest in automated hardware-generation flows that translate high-level algorithmic design intents into synthesizable RTL. Compiler infrastructures such as MLIR, CIRCT, and Calyx provide systematic lowering pipelines that expose scheduling, pipelining, memory management, and resource optimization. However, these flows require substantial training and engineering effort. Nonetheless, a semantic gap in the compilation occurs because standard lowering processes prioritize sequential execution, inadvertently stripping away the high-level data-flow and loop structures required for efficient spatial mapping onto the hardware array. My doctoral research aims to develop a semantic-preserving compiler infrastructure through a principled approach while leveraging existing tools and techniques in intermediate representation and optimization. Our preliminary work shows that LLMs are useful for rapid prototyping of semantic-preserving simple kernels, but compiler-driven flows remain more predictable and efficient for complex, performance-critical accelerators. Future work will develop a complete end-to-end compiler infrastructure for AI model workload specification to efficient hardware generation.**

I. Motivation and Research Problem

FPGA accelerators are attractive for machine-learning workloads because they can exploit application-specific parallelism and efficient data movement. However, producing high-quality RTL still requires expertise in microarchitecture, timing closure, pipelining, memory hierarchy, and board-level resource constraints. Automated hardware generation seeks to reduce this burden by translating high-level algorithmic descriptions into synthesizable hardware.

Compiler-driven flows address this problem through staged lowering from high-level programs to hardware-specific intermediate representations. In frameworks such as Allo, MLIR, CIRCT, and Calyx, tensor or Python-like programs are lowered into explicit loop, memory, and hardware representations before SystemVerilog emission. This makes scheduling, pipelining, bufferization, and resource sharing analyzable and reproducible. In contrast, recent LLMs can generate SystemVerilog directly from natural-language prompts or Python/PyTorch descriptions. This reduces the effort needed to obtain an initial prototype, but it is unclear whether the generated RTL is reliable, efficient, or scalable enough for real accelerator design. A neural-network layer may initially expose tensor reuse, producer-consumer relationships, parallel dimensions, and dataflow boundaries, but after conventional lowering these properties may appear only as low-level loops and memory operations. Once this structure is weakened, the compiler has less information for spatial mapping, memory banking, dataflow construction, and resource-aware exploration.

The central question of my research is: *How can a compiler preserve the semantic structure of AI workloads while progressively lowering them into efficient, resource-aware implementations for reconfigurable heterogeneous SoCs?* This question is broader than generating syntactically correct RTL. Practical accelerators must satisfy correctness, timing, throughput, area, power, and portability constraints under limited LUT, flip-flop, DSP, BRAM, memory-bandwidth, and host–fabric communication budgets. My research studies compiler and LLM-based automation from this perspective: LLMs may reduce design-entry effort, but compiler representations are needed to preserve semantics and optimize performance-critical hardware.

II. Methodology

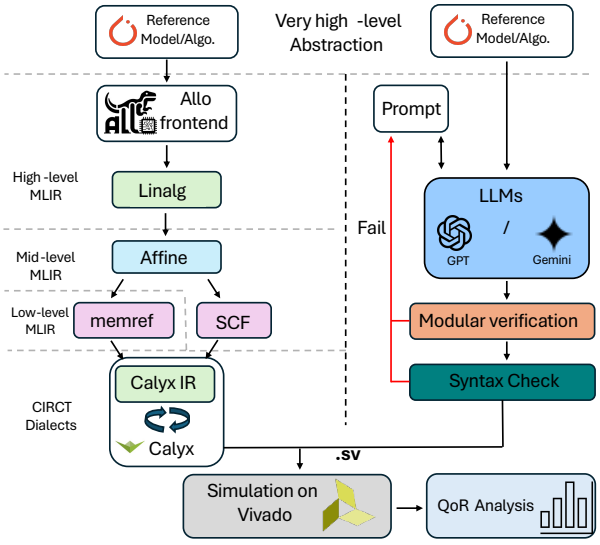

Fig. 1. Compiler-driven vs LLM-driven SV generation.

My current work evaluates two RTL-generation paths from identical functional intent as shown in Fig 1. The compiler-driven path starts from high-level machine-learning kernels written in Python/Allo and lowers them through MLIR-based intermediate representations, including affine, SCF, memref, and hardware-oriented forms, before emitting synthesizable SystemVerilog through CIRCT/Calyx-style lowering. The

LLM-driven path gives equivalent kernel descriptions and constraints directly to commercial LLMs, which generate SystemVerilog without explicit intermediate stages for scheduling, memory layout, resource allocation, or hardware semantics.

Both flows are evaluated using the same downstream FPGA toolchain. The generated RTL is checked for syntax, simulated for correctness, and synthesized in Xilinx Vivado targeting a Zynq-7020 FPGA. The benchmark suite spans different complexity levels: a scalar integer increment kernel, ReLU and GELU activation functions, a 32×32 integer GEMM kernel, and a two-layer feed-forward neural network with input dimension 64, hidden dimension 48, and output dimension 4. The primary metrics are maximum clock frequency, LUT usage, flip-flop usage, DSP utilization, estimated power, correctness, determinism, prompt/debug effort, and scalability.

III. Progress and Preliminary Findings

As a first step, I evaluate whether low-effort LLM-driven RTL generation can replace or complement compiler-driven hardware generation. This study compares two paths from identical high-level machine-learning kernels. The compiler-driven path starts from Python kernels and lowers them through MLIR-based intermediate representations, including affine, SCF, memref, and hardware-oriented forms, before generating SystemVerilog through CIRCT/Calyx-style lowering. The LLM-driven path gives equivalent kernel descriptions and design constraints directly to commercial LLMs, which produce SystemVerilog without explicit intermediate stages for scheduling, memory layout, resource allocation, or hardware semantics.

Both flows are evaluated using the same downstream FPGA toolchain. The generated RTL is checked for syntax, simulated for correctness, and synthesized in Xilinx Vivado targeting a Zynq-7020 reconfigurable heterogeneous SoC. The benchmark suite includes increment, ReLU, GELU, 32×32 GEMM, and a two-layer feed-forward neural network, spanning scalar, tensor, dense arithmetic, and composed workloads. Metrics include Fmax, LUTs, flip-flops, DSPs, power, correctness, determinism, debugging effort, and scalability.

The preliminary results show that LLM-generated RTL is useful for rapid prototyping, especially for simple kernels. Small scalar or activation-style designs can often be generated with few prompt iterations and limited user effort. However, as kernel complexity increases, LLM-generated designs become less reliable and less efficient. For structured workloads such as GEMM and FFNN, LLMs often require repeated prompt-debug cycles and may fail to produce correct or synthesizable floating-point implementations.

The compiler-driven flow produces more predictable and efficient hardware. Across the evaluated kernels, current results indicate that compiler-generated designs achieve approximately $1.7\times$ higher maximum clock frequency than ChatGPT-generated RTL and about $1.4\times$ higher frequency than Gemini-generated RTL on average. The compiler flow also uses substantially fewer LUTs, with reductions of roughly 75% compared with ChatGPT-generated designs and about 50% compared with Gemini-generated designs on average. These trends support the dissertation hypothesis: LLMs can help with interaction and early exploration, but performance-critical accelerator generation requires explicit compiler representations that preserve scheduling, memory, and dataflow semantics.

IV. Expected Contributions and Future Work

The next phase will extend this evaluation into compiler development. I plan to build an end-to-end framework that maps AI workload specifications to HLS-based or RTL-based designs for reconfigurable heterogeneous SoCs under explicit board-level constraints. Semantic information will guide scheduling, tiling, pipelining, memory partitioning, data reuse, precision selection, and resource allocation. These decisions are highly coupled: unrolling may improve throughput but exceed DSP limits, while array partitioning may reduce memory stalls but increase BRAM pressure and routing congestion.

Because final resource usage and clock frequency are known only after synthesis and implementation, exhaustive synthesis is infeasible. My research will investigate pruning, ranking, and feedback-guided exploration under user-specified resource budgets, combining analytical models, lightweight prediction, synthesis reports, and constraint-aware search. I will also study portability by separating algorithmic schedules from board- and SoC-specific resource models.

LLMs will be studied as assistants rather than replacements for the compiler. They may help with front-end design entry, schedule suggestion, HLS pragma generation, compiler-error explanation, testbench construction, and synthesis-feedback-guided refinement. However, the core optimization and code generation will remain compiler-driven so that transformations are explicit, analyzable, and reproducible.

The expected contributions are: (1) a semantics-preserving compilation strategy that retains high-level dataflow, loop, memory, and resource information across lowering stages; (2) a resource-aware design-space exploration framework under board-level constraints; (3) an empirical assessment of LLMs as assistants rather than replacements for compiler infrastructures; and (4) guidelines for choosing direct LLM generation, compiler-driven generation, or hybrid workflows. The final goal is to improve reconfigurable heterogeneous SoC accelerator productivity without sacrificing correctness, timing predictability, resource efficiency, or portability.

References

[1] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019, doi: 10.1145/3282307.

[2] K. Majumder and U. Bondhugula, "HIR: An MLIR-based intermediate representation for hardware accelerator description," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ser. ASPLOS '23, Vancouver, BC, Canada, 2024, pp. 189–201, doi: 10.1145/3623278.3624767.

[3] H. Chen et al., "Allo: A programming model for composable accelerator design," *Proc. ACM Program. Lang.*, 2024.

[4] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. CGO*, 2021.

[5] A. Eldridge et al., "MLIR-based hardware design with CIRCT," in *Proc. WOSET*, 2021.